Rapport d’avancement

DEV3.2

BOUNNI LOUBELO Benicia
2026

Table des matières

★ Introduction.....p02

★ Description des fonctionnalités du programme.....p03

★ Présentation de la structure du programme.....p06

★ Organisation et relations des classes de l'arbre binaire.....p08

★ Structure et exploitation de la table des codes - Convertisseur.....p09

★ Structure et exploitation de la table des codes - Visualisateur.....p11

★ Conclusion.....p13

Introduction

Dans ce projet, il faut réaliser deux outils : un programme pour lire et afficher une image depuis un fichier PIF, et un autre pour enregistrer une image dans ce format.

L'idée est de travailler avec le PIF (Primitive Image Format), un format qui utilise une compression sans perte pour limiter la taille des fichiers, contrairement au format simpliste qu'on avait utilisé avant, qui générait des fichiers trop lourds.

Pour que ca marche, on va se servir de trois tables :

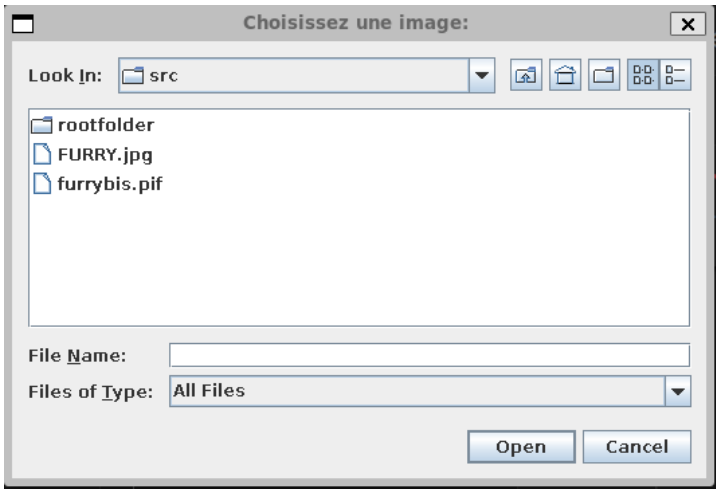
- une **table de fréquence** pour compter combien de fois chaque valeur apparaît ;
- une **table de codes** qui associe a chaque valeur un code binaire plus ou moins court selon sa fréquence (plus une valeur apparaît, plus son code est court);
- et une **table de codes canoniques** pour stocker la table de code de manière compacte dans le fichier.

Description des fonctionnalités du programme

Convertisseur

Le programme permet de sélectionner le fichier à afficher de deux manières :

- **Par ligne de commande** : en spécifiant directement le nom du fichier.
- **Via un explorateur de fichiers** : en navigant pour choisir le fichier.



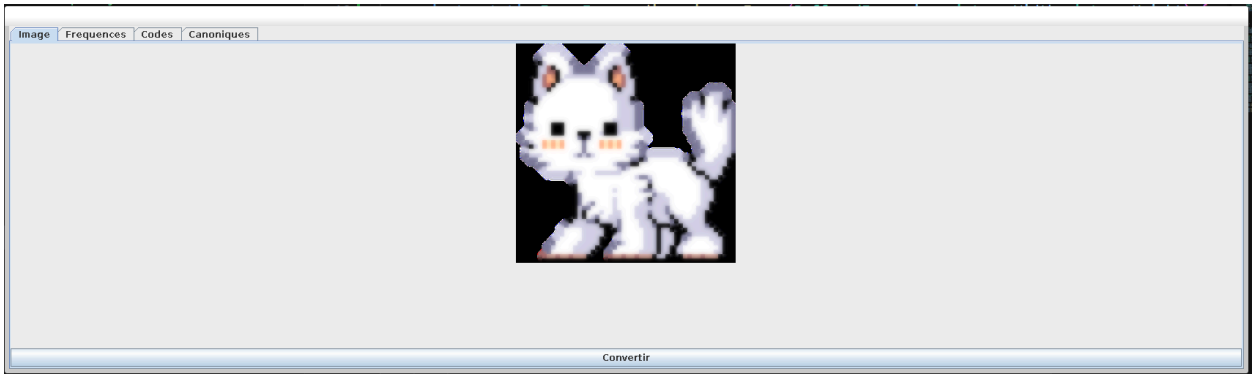
Une fois l'image sélectionnée, elle s'affiche dans une fenêtre. On peut aussi consulter :

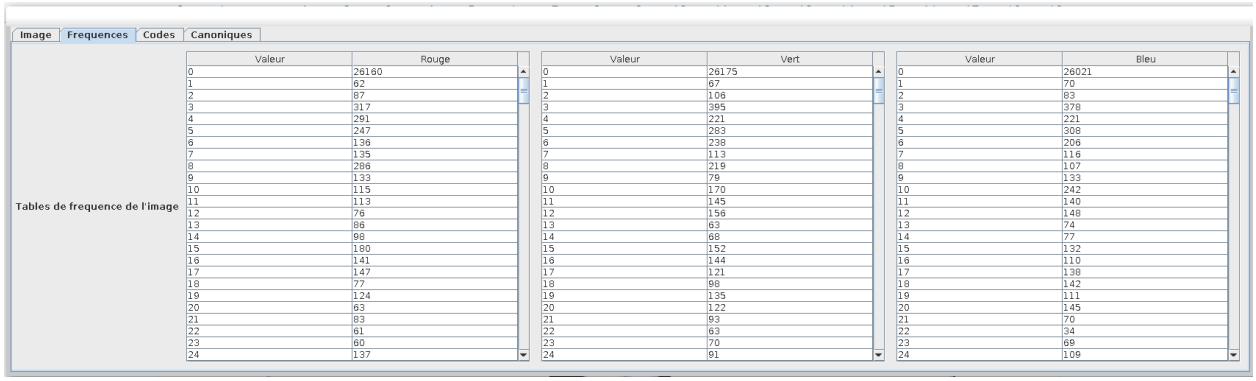
- Les tables de fréquences
- Les tables de codes
- Les tables de codes canoniques

Ces tables sont organisées sous forme d'onglets et s'affichent côte à côte. Si une table est trop longue, il est possible de scroller pour voir la suite.

Un bouton "**Convertir**" permet de lancer la conversion de l'image. Le nom du fichier de sortie peut être :

- Passe en argument dans la ligne de commande.
- Choisis via l'explorateur de fichiers au moment de l'ouverture.

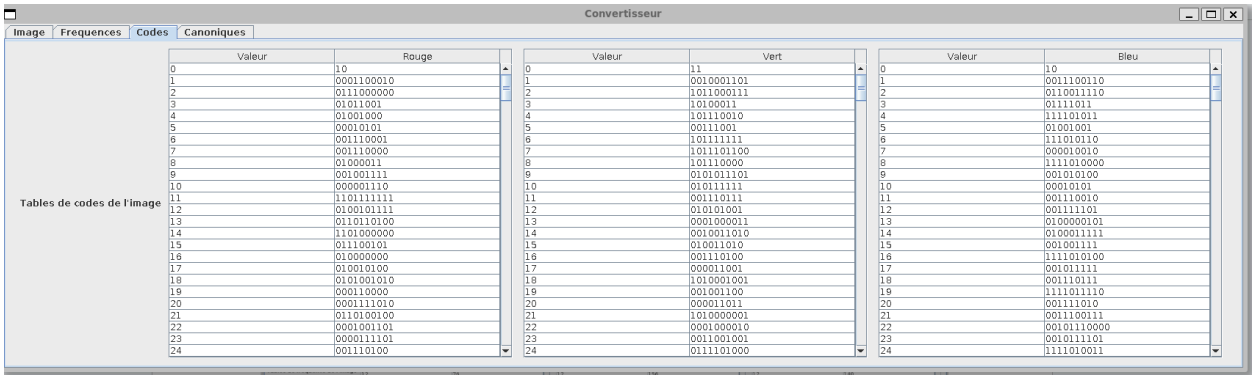




Valeur	Rouge
0	26160
1	62
2	87
3	317
4	291
5	247
6	136
7	135
8	286
9	133
10	115
11	113
12	76
13	86
14	98
15	180
16	141
17	147
18	77
19	124
20	63
21	83
22	61
23	60
24	137

Valeur	Vert
0	26175
1	67
2	106
3	395
4	221
5	283
6	238
7	113
8	219
9	79
10	170
11	145
12	156
13	63
14	68
15	152
16	144
17	121
18	98
19	135
20	122
21	93
22	63
23	70
24	91

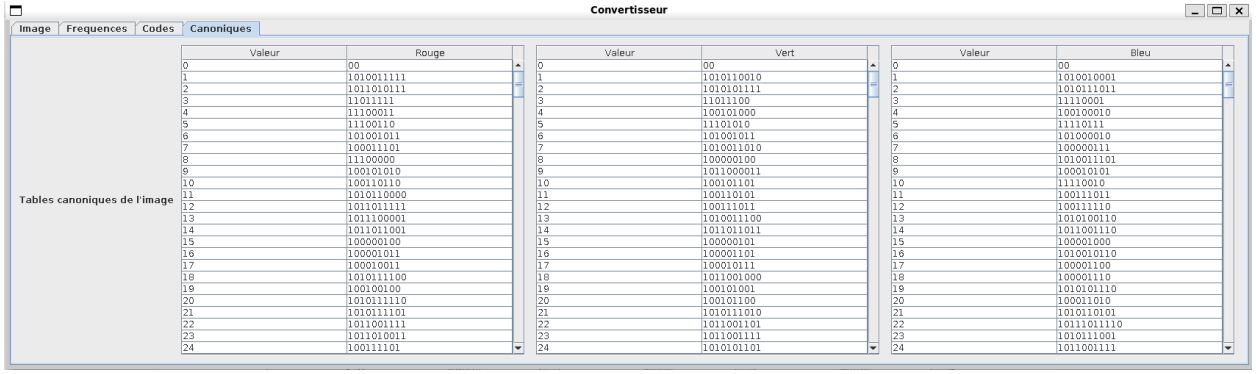
Valeur	Bleu
0	26021
1	70
2	83
3	378
4	221
5	308
6	206
7	116
8	107
9	133
10	242
11	140
12	148
13	74
14	77
15	132
16	110
17	138
18	142
19	111
20	145
21	70
22	34
23	69
24	109



Valeur	Rouge
0	10
1	0001100010
2	0111000000
3	01011001
4	01001000
5	00010101
6	001110001
7	001110000
8	01000011
9	001001111
10	000001110
11	1101111111
12	0100101111
13	0110110100
14	1101000000
15	011100101
16	010000000
17	010010100
18	0101001010
19	000110000
20	0001111010
21	0110100100
22	0001001101
23	0000111101
24	001110100

Valeur	Vert
0	11
1	0010001101
2	1011000111
3	10100011
4	101110010
5	00111001
6	101111111
7	1011101100
8	101110000
9	0101011101
10	010111111
11	001110111
12	010101001
13	0001000011
14	0010011010
15	010011010
16	001110100
17	000011001
18	1010001001
19	001001100
20	000011011
21	1010000001
22	0001000010
23	0011001001
24	0111101000

Valeur	Bleu
0	10
1	0011100110
2	0110011110
3	01111011
4	111101011
5	01001001
6	111010110
7	000010010
8	1111010000
9	001010100
10	00010101
11	001110010
12	001111101
13	0100000101
14	0100011111
15	001001111
16	1111010100
17	001011111
18	001110111
19	1111011110
20	001111010
21	0011100111
22	001011110000
23	0010111101
24	1111010011

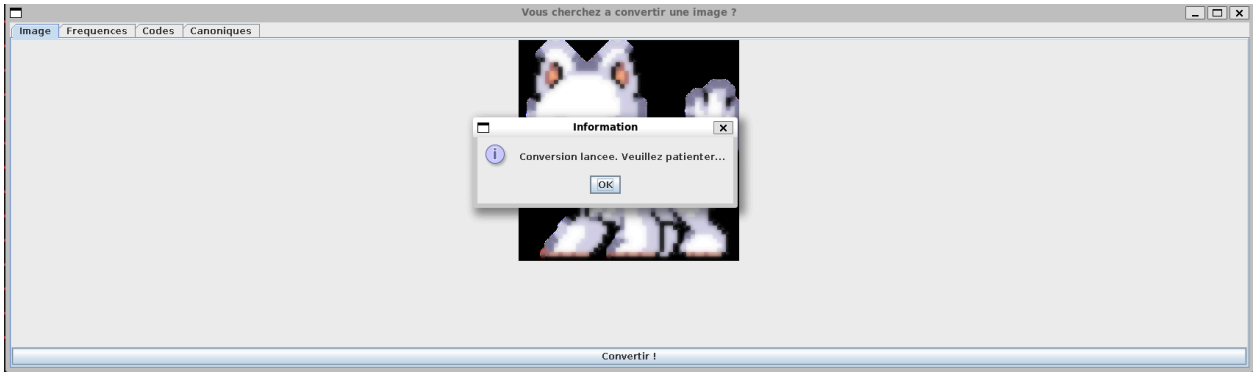


Valeur	Rouge
0	00
1	1010011111
2	1011010111
3	11011111
4	11100011
5	11100110
6	101001011
7	100011101
8	11100000
9	100101010
10	100110110
11	1010110000
12	1011011111
13	1011100001
14	1011011001
15	100000100
16	100001011
17	100010011
18	10101111100
19	100100100
20	1010111110
21	1010111101
22	1011001111
23	1011010011
24	100111101

Valeur	Vert
0	00
1	1010110010
2	1010101111
3	11011100
4	100101000
5	11101010
6	101001011
7	10100011010
8	1000000100
9	10110000011
10	100101101
11	100110101
12	100111011
13	10100011100
14	10110110111
15	1000001101
16	100001101
17	100010111
18	10110011000
19	100101001
20	100101100
21	10101110110
22	10110011101
23	10110011111
24	10101011101

Valeur	Bleu
0	00
1	1010010001
2	1010111011
3	11110001
4	100100010
5	11110111
6	101000010
7	100000111
8	10100011101
9	100010101
10	11110010
11	100111011
12	100111110
13	1010100110
14	1011001110
15	100001000
16	1010010110
17	100001100
18	100001110
19	1010101110
20	100011010
21	1010110101
22	10111011110
23	1010111001
24	1011001111

Une fenêtre de dialogue informe l’utilisateur que la conversion sera entamée

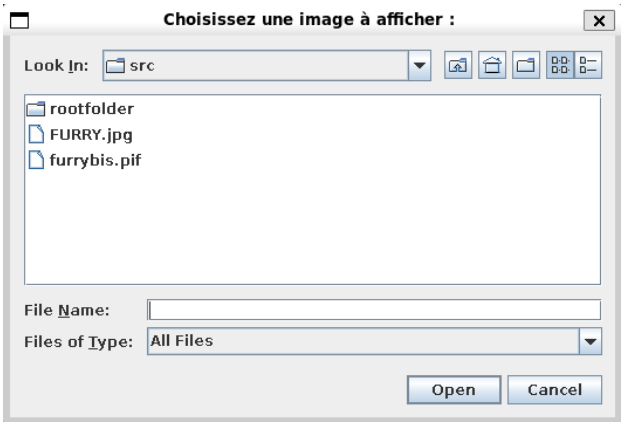


A la fin de la conversion, une fenêtre de dialogue informe l'utilisateur que l'opération est terminée.

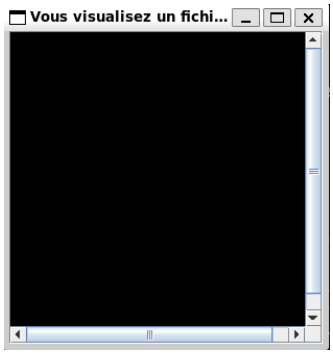
Visualisateur

L'utilisateur peut sélectionner le fichier .pif a afficher de deux façons :

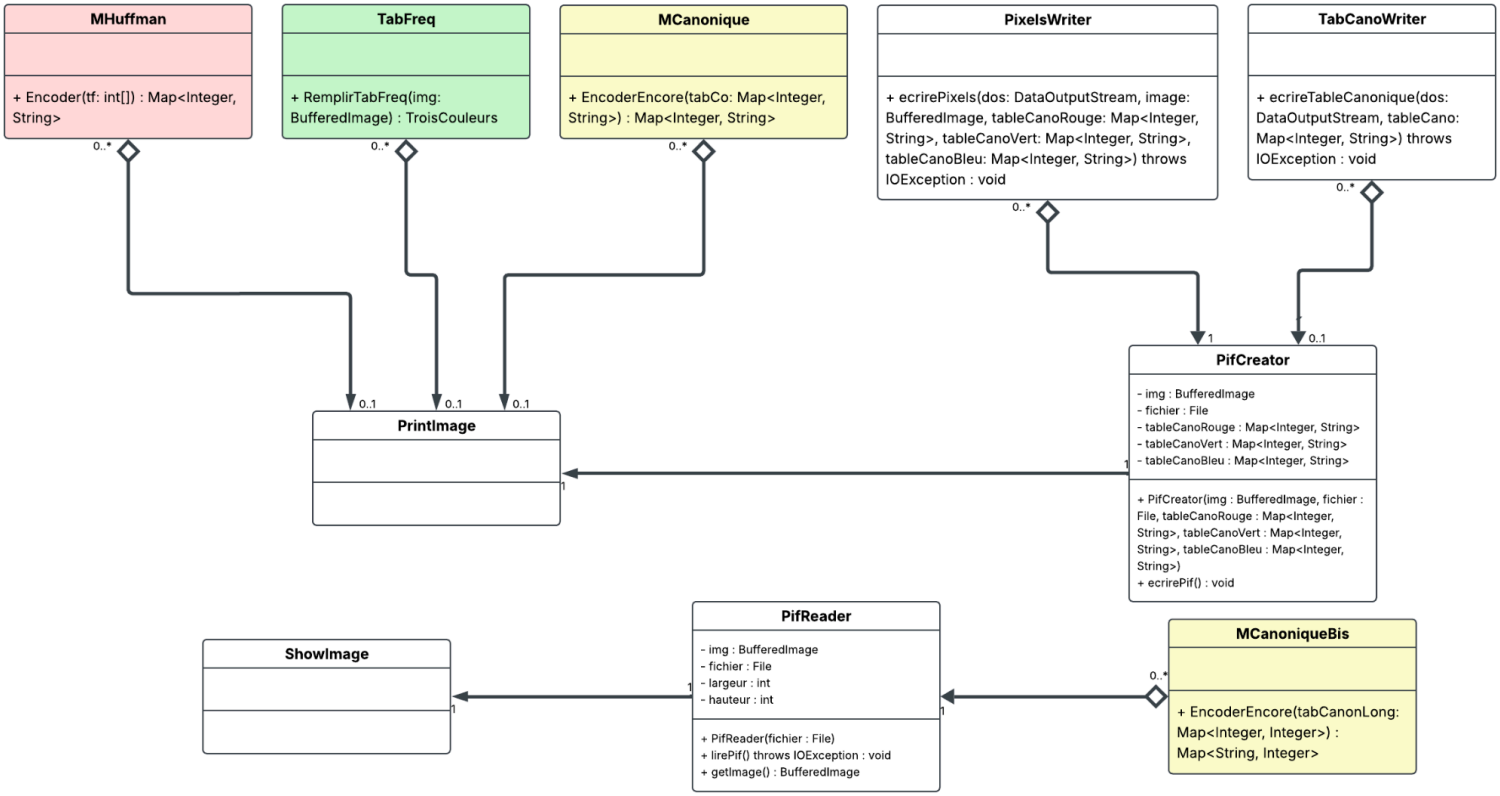
- **Par ligne de commande** : en indiquant directement le chemin du fichier.
- **Via un explorateur de fichiers** : en parcourant les dossiers pour choisir le fichier.



Une fois sélectionné, le fichier .pif s'affiche dans une fenêtre dédiée.



Présentation de la structure du programme



Mon projet repose sur une séparation claire entre la logique métier liée à la compression Huffman et la gestion de l'interface graphique.

J'ai ainsi organisé le programme autour de trois grandes catégories de classes, correspondant directement aux différentes tables décrites dans l'énoncé du format PIF.

Certaines classes existent en double version, avec une variante dédiée au visualisateur (classes suffixées par *bis.java), afin d'adapter le traitement des données au décodage et à l'affichage de l'image.

Classes pour les tables de fréquence (en vert)	Ces classes ont pour rôle de parcourir les pixels de l'image et de compter, pour chaque composante (rouge, vert et bleu), le nombre d'occurrences de chaque valeur comprise entre 0 et 255. Elles produisent, pour chaque composante, une table de

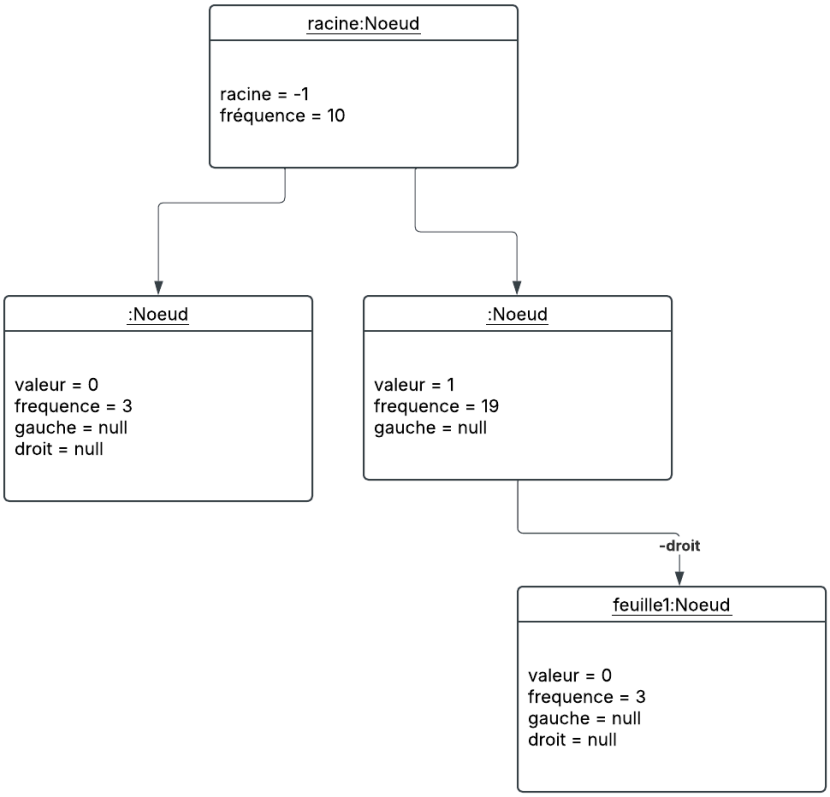
	fréquences, qui constitue la base de l'algorithme de Huffman.
Classes pour les tables de codes (en rouge)	À partir des tables de fréquences, ces classes construisent les arbres de Huffman correspondants. Elles génèrent ensuite les tables de codes initiaux, en associant à chaque valeur un code binaire dépendant de sa position dans l'arbre. Ces tables représentent le résultat direct du codage de Huffman, avant toute optimisation pour le stockage dans le fichier PIF.
Classes pour les tables de codes canoniques (en jaune)	Ces classes récupèrent les tables de codes initiaux et appliquent la transformation décrite dans l'énoncé afin d'obtenir des codes canoniques. Les codes sont triés par longueur puis par valeur, puis reconstruits. Le résultat est une table de codes canoniques où seules les longueurs des codes doivent être stockées, ce qui permet une représentation compacte dans le fichier .pif et une reconstruction sure lors du décodage.
Le reste des classes	Les autres classes du projet servent principalement à : ● gérer l'interface graphique, ● afficher l'image, ● visualiser les différentes tables (fréquences, codes, codes canoniques), ● orchestrer les échanges entre les différentes couches logicielles.

Une fois la logique métier déléguée à ces différentes couches (fréquences, codes, codes canoniques), l'objectif final est d'encapsuler ces traitements pour :

- récupérer les tables associées à une image,
- les afficher dans le convertisseur,
- les utiliser pour encoder ou décoder les pixels lors de la conversion ou de la visualisation.

Au besoin, les différentes tables sont passées en paramètres afin de lancer la conversion vers le format PIF ou d'assurer la reconstruction correcte de l'image dans le visualisateur.

Organisation et relations des classes de l'arbre binaire



La construction et l'exploitation de l'arbre binaire de Huffman reposent sur plusieurs classes distinctes, chacune ayant une responsabilité précise. L'objectif est de séparer clairement la construction de l'arbre, sa structure, et son parcours.

La construction de l'arbre de Huffman s'appuie sur une file de priorité, mais il est important de souligner que :

- la file de priorité n'est pas l'arbre ;
- elle sert uniquement d'outil temporaire pour manipuler, combiner et ordonner les nœuds ;
- l'arbre de Huffman est défini exclusivement par les liens entre les nœuds (références gauche et droite).

Noeud : structure de base de l'arbre

La classe Noeud représente un nœud d'un arbre binaire de Huffman.

Elle constitue l'élément central de la structure de données.

Un nœud peut être :

- une feuille, associée à une valeur de pixel et à sa fréquence ;
- un nœud interne, qui ne représente aucune valeur mais combine deux sous-arbres.

Chaque nœud contient :

- une valeur (uniquement pour les feuilles),
- une fréquence,
- une référence vers un fils gauche,
- une référence vers un fils droit.

La classe implémente l'interface Comparable, ce qui permet de trier les nœuds par fréquence lors de leur insertion dans une file de priorité. Ce mécanisme est indispensable pour appliquer correctement l'algorithme de Huffman, qui fusionne toujours les nœuds de plus faible fréquence en priorité.

Les relations gauche/droite entre les instances de `Noeud` forment l'arbre binaire lui-même.

`ArbreHuffman` : construction de l'arbre

La classe ArbreHuffman est responsable de la construction de l'arbre binaire de Huffman à partir d'une table de fréquences.

Son fonctionnement se déroule en deux étapes principales :

1. **Création d'une file de priorité**
   Pour chaque valeur dont la fréquence est non nulle, un nœud feuille est créé et inséré dans une file de priorité.
   Cette file permet d'accéder rapidement aux nœuds de plus faible fréquence.
2. **Fusion progressive des nœuds**
   Tant que la file contient plus d'un élément, les deux nœuds ayant la plus faible fréquence sont retirés, fusionnés dans un nouveau nœud interne, puis réinsérés dans la file.

Ce nouveau nœud devient le parent des deux nœuds extraits.

À la fin du processus, la file ne contient plus qu'un seul élément :
la racine de l'arbre de Huffman.

La file de priorité n'est donc utilisée que comme outil de construction. Une fois l'arbre créé, seule la racine est conservée, et l'arbre est entièrement représenté par les liens entre les nœuds.

ParcoursArbre : exploitation de l'arbre

La classe ParcoursArbre permet de parcourir l'arbre de Huffman afin de générer la table des codes binaires.

J'ai eu l'idée d'utiliser un parcours en profondeur (DFS) récursif :

- un déplacement vers le fils gauche ajoute un 0 au code,
- un déplacement vers le fils droit ajoute un 1.

Lorsque le parcours atteint une feuille, le code binaire accumulé est associé à la valeur portée par cette feuille.
Ce mécanisme garantit que :

- chaque valeur reçoit un code unique,
- les valeurs les plus fréquentes obtiennent des codes plus courts.

MHuffman : chef d'orchestre

La classe MHuffman joue un rôle de façade pour l'encodage Huffman.

Elle coordonne les différentes étapes :

1. construction de l'arbre via ArbreHuffman,
2. parcours de l'arbre via ParcoursArbre,
3. génération finale de la table de codes de Huffman.

Cette classe permet ainsi de transformer directement une table de fréquences en une table de codes binaires, sans exposer les détails internes de la construction et du parcours de l'arbre.

Structure et exploitation de la table des codes - Convertisseur

Forme de la table des codes

Dans le convertisseur, la table des codes est produite à partir d'une table de fréquences associée à une composante de couleur. Cette table prend la forme d'une structure de type dictionnaire (Map<Integer, String>), où chaque clé correspond à une valeur de pixel (comprise entre 0 et 255) et chaque valeur associée est un code binaire de longueur variable, généré par l'algorithme de Huffman.

Génération de la table des codes

La classe MHuffman est responsable de la construction de cette table. À partir de la table de fréquences, elle construit un arbre binaire de Huffman, puis le parcourt en profondeur. Lors de ce parcours, chaque feuille de l'arbre, représentant une valeur de pixel, se voit attribuer un code binaire constitué de la succession de 0 et de 1 correspondant aux déplacements gauche et droite depuis la racine.

Cette représentation garantit que :

- chaque valeur possède un code unique,
- les valeurs les plus fréquentes sont associées à des codes plus courts,
- la table obtenue est suffisante pour encoder l'image.

La table de codes ainsi générée constitue une étape intermédiaire, avant la transformation en table de codes canoniques.

Mécanisme d'exploitation pour l'encodage

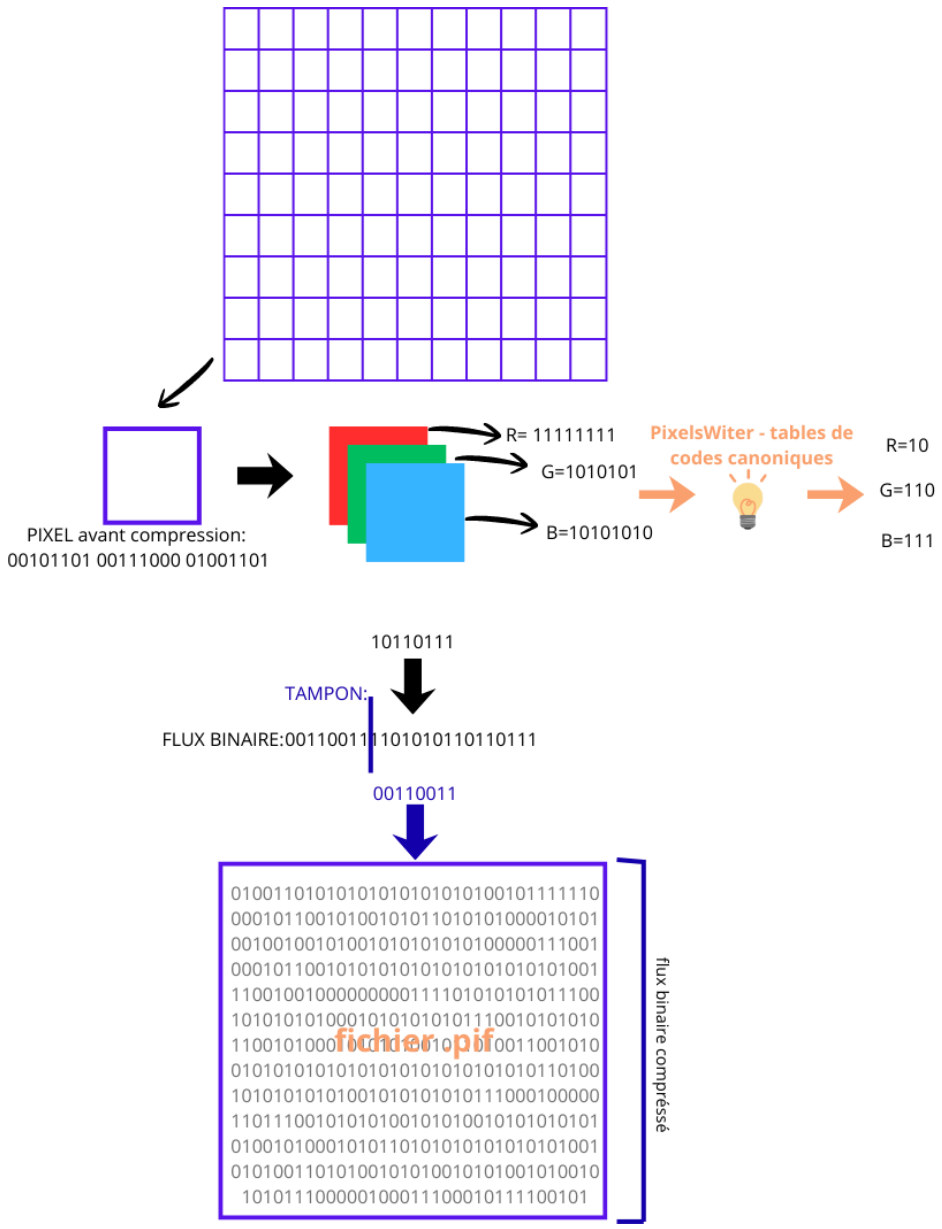
Lors de l'encodage de l'image, les tables de codes canoniques sont utilisées pour convertir les pixels en un flux binaire compressé. La classe PixelsWriter se charge de ce traitement.

Pour chaque pixel de l'image, les composantes rouge, verte et bleue sont extraites, puis remplacées par leurs codes canoniques respectifs. Les trois codes obtenus sont ensuite concaténés afin de former une séquence binaire représentant le pixel.

Ces séquences binaires sont enchaînées pour l'ensemble des pixels de l'image, formant un flux continu de bits. Ce flux est ensuite découpé en octets : les bits sont accumulés dans un tampon jusqu'à atteindre 8 bits, puis écrits dans le flux de sortie. Si le dernier octet n'est pas complet, il est complété par des zéros avant écriture.

Ce mécanisme permet d'encoder l'image sans contrainte d'alignement par pixel, conformément au format PIF, et d'obtenir une compression sans perte fondée sur des codes binaires de longueur variable.

Schéma explicatif reprenant les valeurs données en exemple dans l'énoncé:



Structure et exploitation de la table des codes - Visualisateur

Forme de la table des codes

Dans le visualisateur, la table des codes n'est pas constituée des codes binaires eux-mêmes, mais des longueurs de codes. Ainsi:

- Chaque clé de la table correspond à une valeur de pixel (0 à 255).

- Chaque valeur associée représente la longueur du code binaire canonique pour cette valeur.
  La table est donc un Map<Integer, Integer> (valeur → longueur), ce qui suffit pour reconstruire les codes binaires canoniques de manière déterministe.

Cette représentation est compacte et évite de stocker tous les codes binaires, tout en conservant l'information nécessaire pour le décodage.

Mécanisme d'exploitation pour le décodage

Le décodage repose sur la transformation de cette table de longueurs en codes canoniques effectifs, réalisée par MCanoniqueBis :

1. La table de longueurs est triée par longueur, puis par valeur (CodeLengthSorterBis).

2. Les codes canoniques sont générés de manière déterministe (TabCaFinaleBis) : chaque valeur de pixel obtient un code binaire unique respectant l'ordre croissant des longueurs.

3. Le flux binaire compressé du fichier PIF est lu bit par bit. Pour chaque composante de couleur :
   - Les bits sont accumulés dans un tampon jusqu'à former un code présent dans la table canonique,
   - Ce code permet de retrouver la valeur de pixel correspondante,
   - La valeur est utilisée pour reconstruire l'image pixel par pixel.

Ainsi, la forme “longueur de code” de la table dans le visualisateur est directement exploitable pour générer les codes binaires nécessaires au décodage, contrairement au convertisseur où l’on devait d’abord trouver la longueur du code.

Conclusion personnelle

Étant donné que ce projet a été réalisé uniquement par moi, j’ai eu peu d’occasions d’échanger ou de réfléchir avec mes camarades. Cependant, cette expérience a été réellement formatrice et m’a permis d’apprendre énormément en très peu de temps. J’ai été impliquée dans tous les aspects du projet (le code, la logique, le makefile, ainsi que la rédaction du rapport) et j’ai pu mener le projet de A à Z. Cette autonomie m’a forcée à développer ma capacité à organiser un projet entier, à résoudre les problèmes de manière créative.

J’ai aussi mieux compris le processus de développement dans son ensemble.

Si je devais recommencer, je prendrais davantage de temps pour planifier certaines étapes en amont et j’utiliserais de meilleures méthodes pour tester systématiquement les différentes parties du code.

D’un autre côté, le temps consacré à la compréhension de l’énoncé et à la conception m’a permis d’aborder le codage de manière plus structurée et avec moins de tâtonnements. En revanche, j’ai rencontré des difficultés lors de l’implémentation, notamment pour intégrer correctement les différentes classes entre elles.

Si le découpage algorithmique était clair, sa traduction en code s’est parfois révélée fragile.

Je suis très fière du travail accompli et des compétences développées tout au long de ce projet.